

KUGGLE 스터디 3주차 · LLM 세션

어텐션과 트랜스포머

GPT의 T는 무엇인가 — 15분 안에 훑는 핵심 구조

컴퓨터가 사람의 말을 알아듣게 된 결정적인 전환점이 2017년에 있었습니다. 오늘은 그 전환점을 만든 두 부품, 어텐션과 트랜스포머가 각각 어떤 일을 맡고 있는지 살펴봅니다.

한 단어가 문장 끝에서 뜻을 뒤집는다

bank를 읽는 시점에는 road도 river도 아직 나오지 않았습니다

마지막 단어가 ROAD일 때

I arrived at the **bank** after crossing the road.

→ 돈을 맡기는 은행

길을 건넜다는 말이 나오는 순간, bank는 길가에 서 있는 건물이 됩니다.

마지막 단어가 RIVER일 때

I arrived at the **bank** after crossing the river.

→ 강가의 둑

단어 하나가 바뀌었을 뿐인데 같은 bank가 지형을 가리키게 됩니다.

사람은 0.1초 만에 구분합니다. 뜻을 결정하는 단서가 그 단어보다 한참 뒤에 있어도 문제가 없습니다. 컴퓨터에게는 이것이 오랫동안 어려운 일이었습니다.

예전에는 단어 하나에 숫자 한 벌

뜻이 여러 개여도 그 단어에 붙는 숫자는 언제나 하나였습니다

단어 하나 = 숫자 한 벌

예를 들어 “차”라는 단어는 [0.21, -1.34, 0.88, 0.07, -0.42, ...] 같은 숫자 목록 하나로 표현됩니다.

단어를 이렇게 숫자 목록으로 바꾼 것을 **임베딩**이라고 부릅니다. 비슷한 뜻을 가진 단어끼리 비슷한 숫자를 갖도록 학습시킵니다.

문제는 이 숫자가 어떤 문장에 쓰이든 **언제나 똑같다**는 점이었습니다.

나는 어제 차를 마셨다

마시는 차

앞을 우려낸 음료를 가리킵니다.

“마셨다”가 뜻을 정해줍니다.

동생이 차를 타고 갔다

타는 차

바퀴가 달린 탈것을 가리킵니다.

“타고”가 뜻을 정해줍니다.

그런데 두 문장의 “차”는 기계에게 완전히 같은 숫자였습니다. 사전을 외우면서 한 단어에 뜻을 하나만 적어둔 사람인 셈입니다.

그리고 문장은 한 단어씩 순서대로 읽었다

RNN이라고 부르는 방식입니다. 사람이 읽는 과정과 닮았지만 두 가지가 발목을 잡았습니다

한 번에 한 단어씩, 기억을 넘기며

나는 → 어제 → 차를 → 마시며 → 읽었다

한 단어를 읽을 때마다 지금까지의 내용을 하나의 "기억"에 눌러 담아 다음 칸으로 넘깁니다. 그래서 마지막 칸이 보는 것은 문장 전체가 아니라 여러 번 압축을 거친 요약본입니다.

① 끝에 가면 앞을 잊는다

문장이 길어질수록 초반 정보가 눌러 사라집니다.

긴 글의 첫 문장을 마지막 문장까지 온전히 들고 가기 어려웠습니다.

100명이 이어서 하는 귓속말 전달 게임과 같은 상황입니다.

② 순서를 기다려야 한다

앞 단어의 계산이 끝나야 다음 단어를 볼 수 있습니다.

GPU는 수천 개를 동시에 계산할 수 있는데 그 능력을 쓸 수가 없습니다.

학습이 느리니 모델을 크게 키우는 데 한계가 생깁니다.

두 문제 모두 "순서대로 읽는다"는 전제에서 나왔습니다. 그 전제를 버린 것이 트랜스포머입니다.

정리하면, 벽은 세 개였다

지금부터 나올 이야기는 전부 이 세 칸을 채우는 과정입니다

① 느리다

순서대로만 읽으니 문장을 한꺼번에 처리할 수 없습니다.

GPU의 병렬 계산 능력을 거의 쓰지 못합니다.

학습이 느려 모델을 키우는 데 한계가 생깁니다.

3막에서 해결

② 멀면 놓친다

멀리 떨어진 단어끼리의 연결이 중간에서 끊깁니다.

100번째 단어가 1번째 단어를 보려면 100단계를 거칩니다.

문장이 길수록 앞부분이 더 많이 사라집니다.

3막에서 해결

③ 문맥을 모른다

단어의 숫자가 문장과 무관하게 고정되어 있습니다.

같은 단어는 어떤 문장에 있든 언제나 같은 뜻입니다.

bank도 “차”도 구분하지 못합니다.

2막에서 해결

오늘 발표는 이 세 칸을 하나씩 지우는 이야기입니다.

먼저 ③ 문맥 문제를 어텐션이 풀고, ①과 ②를 트랜스포머가 함께 풀어줍니다.

아이디어는 시험공부에서 왔다

우리는 같은 페이지 안에서도 문장마다 다른 무게로 읽습니다

교재 한 페이지

자연어 처리 모델은 오랫동안 문장을 왼쪽부터 순서대로 읽었다.

이 방식은 사람이 글을 읽는 과정과 닮아 자연스러워 보였다.

핵심은 문장을 한꺼번에 볼 수 있느냐였다.

2014년에는 인코더와 디코더를 잇는 여러 시도가 이어졌고,
정보가 압축되면서 사라지는 문제가 계속 지적되었다.

멀리 떨어진 단어를 직접 연결하는 것이 관건이었다.

여러 연구가 이 문제를 조금씩 다른 방식으로 다루었다.

2017년, 어텐션만으로 충분하다는 결론이 나왔다.

사람이 읽는 방식

제목, 밑줄 친 문장, 숫자에 **더 많은 주의**를 기울입니다.

나머지는 눈으로 훑고 지나갑니다.

문장마다 무게가 다르고, 그 무게는 읽는 목적에 따라
매번 달라집니다.

어텐션은 이 무게를 **숫자로 계산**합니다.

왼쪽 페이지에서 노란 줄 세 개만 남기고 나머지를 지
워도 글의 뼈대는 남습니다. 어텐션이 하는 일이 정확
히 이것입니다.

어텐션은 이 “선택적 집중”을 숫자로 만든 것입니다. 이름이 어려워 보이지만 아이디어는 이게 전부입니다.

오늘 딱 한 줄만 기억한다면

어텐션이 하는 일은 이 한 문장이 전부입니다

지금 이 단어의 뜻을 정할 때, 문장 속 다른 단어들을 얼마씩 참고할지 정하는 배분율

예를 들면

새로운 bank = river 41% + crossing 16% + bank 13% + arrived 10% + ...

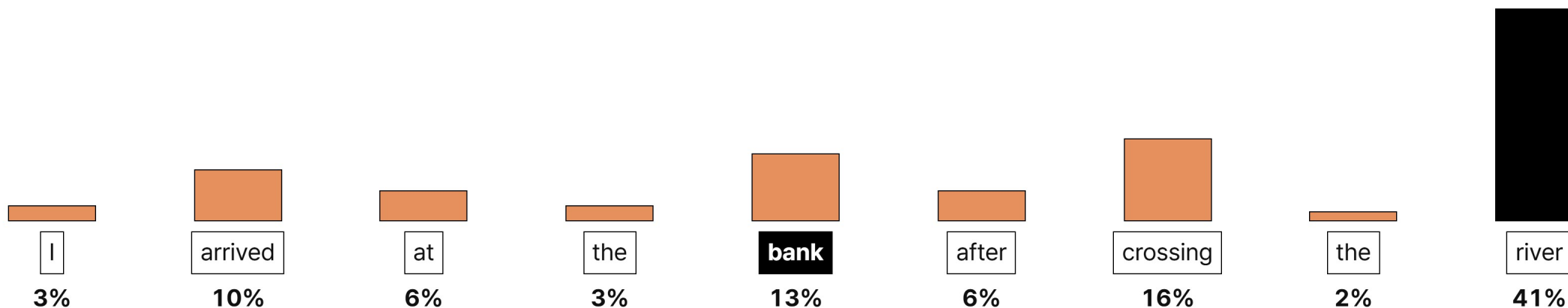
river에 41%, crossing에 16%를 배분했다는 뜻입니다. 비율의 합은 언제나 100%가 되고, 이 비율대로 각 단어의 정보를 섞은 결과가 bank의 새로운 뜻이 됩니다.

중요한 단어는 많이, 상관없는 단어는 조금. 이 배분율을 문장마다 새로 계산하는 것이 어텐션입니다.

bank가 river를 보는 순간

문장 전체를 한 번에 훑고 단어마다 점수를 매깁니다

BANK가 각 단어를 참고하는 비율 — 설명용 예시 수치



bank가 문장 안의 모든 단어와 자기를 비교해 점수를 매기고, 그 점수를 확률처럼 바꾼 값입니다. river가 41%로 가장 높으므로 bank의 숫자는 river 쪽으로 끌려가 "강둑 bank"로 다시 계산됩니다.

문장 안에서 단어들이 서로를 본다고 해서 이것을 셀프 어텐션이라고 부릅니다. 벽 ㉓ 문맥 문제가 여기서 풀립니다.

점수를 매기는 방법에 붙은 이름

Q, K, V — 유튜브에서 영상을 찾는 과정과 구조가 똑같습니다

Q · QUERY	K · KEY	V · VALUE
검색창에 친 말 “내가 지금 뭐가 필요하지?” 뜻을 정하려는 단어가 던지는 질문입니다.	각 영상의 제목과 태그 “난 이런 걸 가지고 있어” 각 단어가 자신을 설명하는 이름표입니다.	영상의 실제 내용 점수만큼 가져올 알맹이 비율대로 섞여 들어가는 정보 그 자체입니다.

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

Q와 K를 맞춰본 점수를 확률로 바꾸고(softmax), 그 비율대로 V를 섞는다는 뜻입니다.

논문을 열면 이 식이 나옵니다. 오늘 이해하지 않으셔도 됩니다.

검색어(Q)와 태그(K)를 맞춰보고, 맞는 만큼 내용(V)을 가져옵니다. 위 카드 세 장이 곧 저 수식입니다.

트랜스포머는 어텐션을 쌓은 것이다

층이 올라갈수록 더 추상적인 관계를 봅니다

층을 쌓는다	한 층 안에서도 여러 명이 동시에 읽는다
최상층 · 이 문장이 하려는 말 4층 · 문장 전체의 의미 3층 · 구와 절의 관계 2층 · 단어 뜻의 미세 조정 1층 · 어순과 문법 관계 아래층의 결과를 위층이 다시 읽습니다. 2017년 논문은 6층이었습니다.	한 명은 주어와 동사의 관계를, 한 명은 단어 뜻의 유사성을, 또 한 명은 멀리 떨어진 단어를 봅니다. 각자 본 것을 모아 합칩니다. 이것을 멀티헤드 어텐션이라고 부릅니다. 관점 하나만으로는 문장의 여러 겹을 다 잡아내지 못하기 때문입니다. 빠진 부품 두 개 어텐션은 순서를 모릅니다. 그래서 단어마다 좌석 번호표를 붙여줍니다 — 위치 인코딩. 그리고 읽는 쪽(인코더)과 써나가는 쪽(디코더)이 나뉘어 있습니다. 쓸 때는 뒷부분을 가리고 씁니다.

트랜스포머는 거창한 이름이지만, 어텐션을 층으로 쌓고 부품 몇 개를 덧댄 구조입니다.

벽 두 개가 한꺼번에 무너졌다

멀리 보는 것도, 빨리 보는 것도 어텐션 하나로 해결됩니다

항목	RNN	셀프 어텐션
단어와 단어 사이 최단 거리	n 단계	1 단계
순서대로 기다려야 하는 연산	n 번	1 번
문장 전체 동시 처리	불가능	가능

8 학습에 쓴 GPU 장수	3.5일 학습 시간	28.4 BLEU · 당시 최고 기록
--------------------------	----------------------	--------------------------------

RNN은 100번째 단어가 1번째 단어를 보려면 100단계를 거쳐야 합니다. 어텐션은 한 단계에 직접 봄니다. 벽 ② 해결.
 그리고 순서를 기다릴 필요가 없으니 문장 전체를 한꺼번에 GPU에 넘길 수 있습니다. 벽 ① 해결.

Vaswani et al., Attention Is All You Need (2017) — Table 1, Table 2. n = 문장 길이. BLEU는 WMT 2014 영어→독일어 기준.

그래서 지금의 LLM이 가능해졌다

병렬로 돌아가니 키울 수 있었고, 키웠더니 성능이 계속 올라갔습니다

왜 갑자기 커졌나

병렬로 돌아간다 → 모델을 키울 수 있다 → 키웠더니 잘한다 → 계속 키운다

병렬 처리가 가능해지자 모델 크기를 키우는 것이 현실적인 선택지가 되었고, 키울수록 성능이 올라간다는 사실이 GPT-3 이후 확인되면서 연구가 한 방향으로 몰렸습니다.

인코더만 · BERT

읽고 판단하는 일

분류, 검색, 문장 이해

문장 전체를 한꺼번에 보고 뜻을 파악합니다.

디코더만 · GPT, Claude

이어서 써나가는 일

대화, 글쓰기, 코드 생성

다음 단어를 하나씩 예측하는 방식입니다.

둘 다 · T5, BART

읽고 다시 쓰는 일

번역, 요약

원문을 이해한 뒤 새 문장을 만들어냅니다.

GPT = Generative Pre-trained Transformer. 마지막 T가 오늘 배운 그것입니다.

오늘의 한 줄

다음 시간엔 이 구조를 어떻게 학습시키는지 봅니다

어텐션은 뜻을 문맥으로 다시 계산하고, 트랜스포머는 그 계산을 병렬로 쌓는다.

어텐션	트랜스포머	LLM
뜻을 문맥으로 다시 계산한다 문장 안의 다른 단어를 얼마씩 참고할지 정하고, 그 비율대로 섞습니다.	그 계산을 여러 층으로 쌓는다 층이 올라갈수록 더 추상적인 관계를 보고, 한 층 안에서 여러 관점이 동시에 곱니다.	이 구조를 아주 크게 키운 것 병렬 처리가 가능해진 덕에 키울 수 있었고, 키우니 성능이 따라 올라갔습니다.

오늘은 구조까지입니다. 구조가 있다고 바로 똑똑해지지는 않습니다. 이 구조를 어떻게 학습시키느냐가 다음 시간 주제입니다.